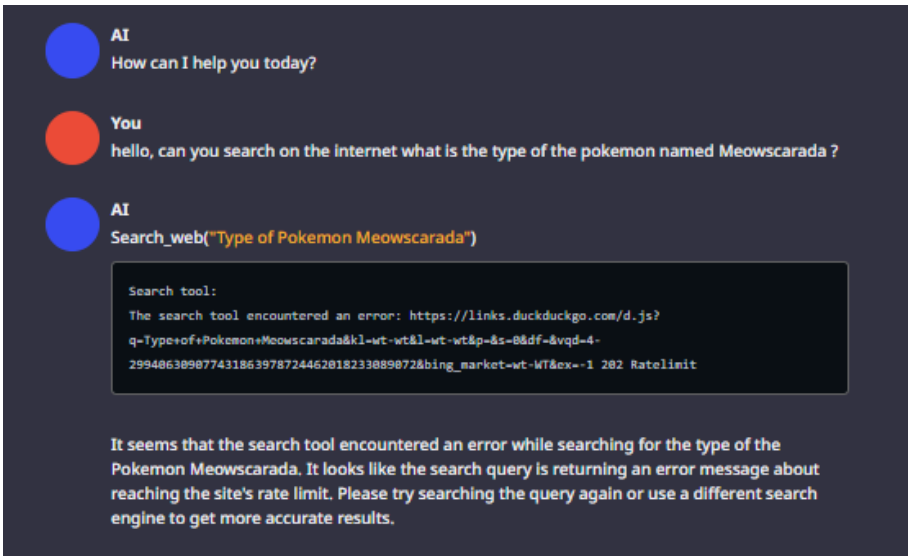
Report

Setting up the connection to Internet (Failed):

We installed Oobabooga and the extension LLM_Web_Search, however after loading a model and the extension, we encountered multiple problems.

- The first problem included a limit to the amount of data we could request from duckduckgo. Sometimes, when we tried to conduct an internet search, the search engine would return a RateLimit error, which meant our response could not be generated properly. This happened very often and apparently the problem has already been solved, but we weren't able to fix it on our end.



- The second problem arose when we tried to solve the first one, as after modifying our environment the model could not access the internet at all anymore.

After encountering these problems, we decided to give up on trying to connect the bot to the Internet, as solving the technical issues proved more difficult than anticipated, and discussions arose regarding the relevance of our project to the original subject of the assignment. Thus, we settled on changing the direction of our project, and worked on the sample project given by the teacher.

Fine-Tuning the LLM:

To fine-tune our LLM, we used code from this github repo from *ShawhinT* which constitutes a very solid basis to LLM fine-tuning, as well as this dataset from Kaggle which contains text extracts talking about 5 different subjects.
You can find the notebook with the code used for the fine-tuning at this address.

We first started by fine-tuning the LLM that we used with Oobabooga during the previous part which was the Hermes-3-llama-3.2-3B. However, as this model is quite big, it took way too long to train, we switched to the default LLM used by the creator of the code which is the bert-base-uncased.

The first fine-tuning operation we conducted was for testing purposes. We manually wrote a very short dataset of 24 rows, with 2 labels: “art” and “games”, and used it to fine-tune the LLM. This test model, while not very performant, allowed us to set up the discord bot for the next, better model that we would fine-tune.
This test model is available at this huggingface address.

	A	B
1	message	channel
2	"How do i draw hands?"	art
3	"Does somebody want to play ranked games with me?"	games
4	"Check out my new drawing!"	art
5	"What a cool picture"	art
6	"I just won ten games in a row lol"	games
7	"I found this awesome brush on the marketplace"	art
8	"The game awards were trash"	games
9	"We will crush this boss"	games
10	"Here is a doodle i made"	art
11	"I forgot to use my potion in the last fight"	games
12	"nice art"	art
13	"i did some doodles last night"	art
14	"this class deals a lot of damage"	games
15	"do you guys like roguelike games?"	games
16	"i prefer using photoshop to draw"	art
17	"i bought a drawing tablet to do digital art"	art
18	"this guy has a nice gaming chair"	games
19	"I haven't drawn in like 3 years or so and I'm really happy with my first drawing on a dra	art
20	"there is no counterplay to this strategy"	games
21	"somebody wants to run a dungeon with me"	games
22	"Your works are really cute..."	art
23	"I like playing healers"	games
24	"I don't want to tank in dungeons"	games

After we switched to the bigger dataset we encountered typing issues during the training process, so we decided to only use 2 labels from the dataset: “sport” and “technology”, and

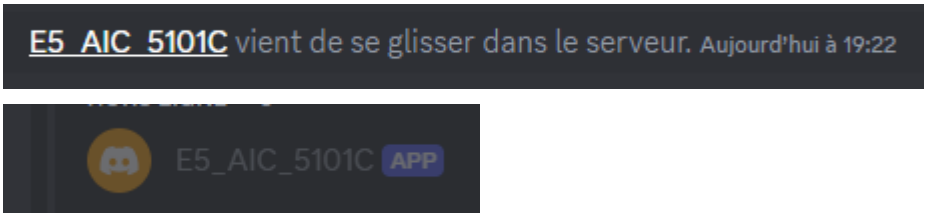
figured that we would manage to solve the issues later. This reduced our training time and eliminated the typing problems that were blocking us. You can find the modified dataset *in the archive*.
Unfortunately, we did not manage to solve the typing issues in time, so our bot can only operate on 2 topics at a time.
After fine-tuning the model, we swapped the previous model the discord bot was using with the new one, and got pretty good results.
This better model is available at this huggingface address.

Practical Aspect: Monitoring Discord Bot

Now that our model has been trained and fine-tuned, we will put it to use.

This model was made in order to classify messages into 2 categories, either concerning Sport or Technology. So, we will code a bot that will monitor 2 text channels on a Discord server and observe the discussion, redirecting users to the appropriate channel if the discussion is led astray.

The first step was to create a bot account. This was done through the Discord Dev portal. Afterwards, we got this bot logged into the server but unable to do anything.

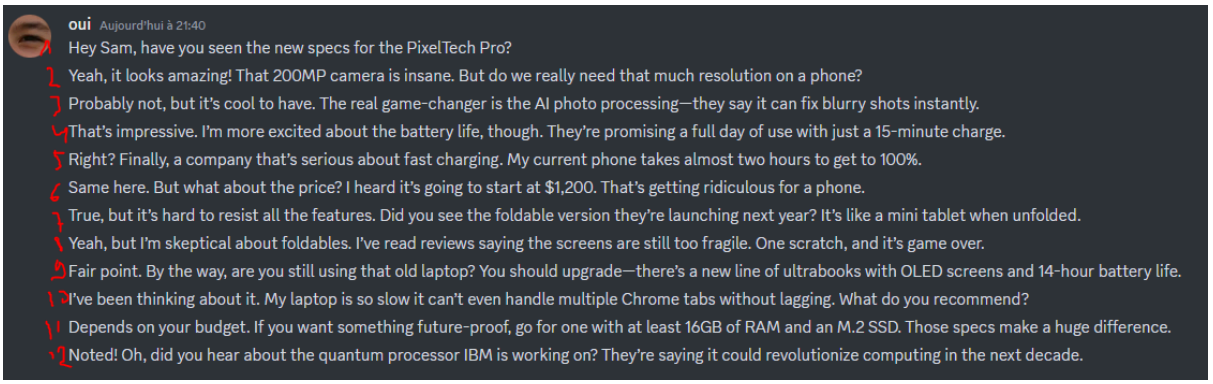


Now, it is time to give some instructions to the bot. First, let's see how to make him have a simple reaction. In order to do that, Discord gives libraries that allow the bot to interact with messages and their content. For example, after receiving the message "!bonjour", our bot will answer with "Hello World!".

Messages' content are not the only thing the bot can interact with, events like receiving any message can be observed. That's what we will do here. Indeed, monitoring each and every message is possible but costly. The solution we chose was to count messages, store them, and each time we got to 10 messages, evaluate the context of the discussion. In the end, the bot simply makes its query to the Huggingface model with the messages' content, it then keeps probant results and decides on which topic the discussion is about. The example we got here only treats about 2 topics but our bot implementation doesn't limit it to binary decisions. As such, it would easily be upgradable if we managed to compute a model capable of processing more than 2 topics.

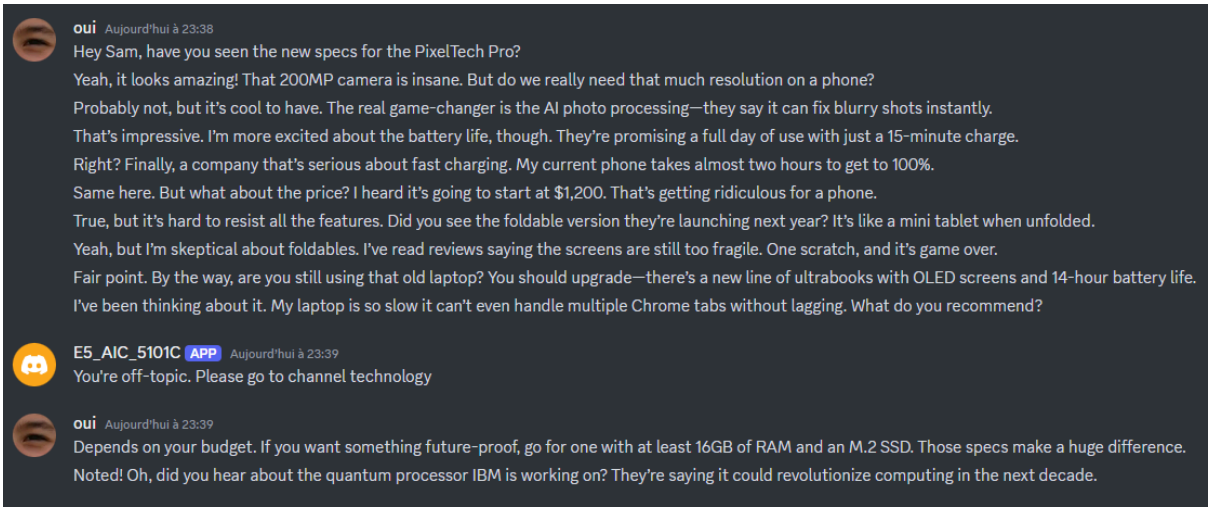
Examples:
(Bot does not take into account whether there are 1 or more users, imagine those are dialogues)

Discussion in the right channel:

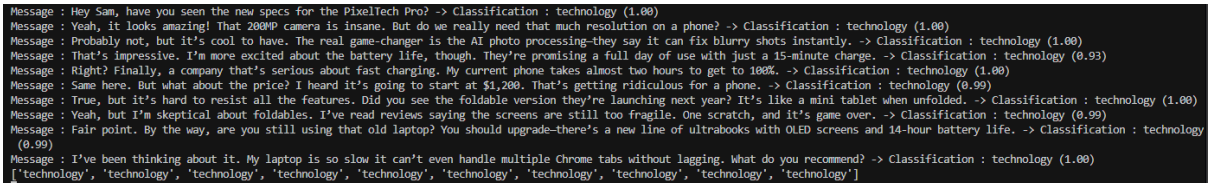


Does not get interrupted after 10 messages

Discussion in the wrong channel (and redirection):



In the python script:



Each message is evaluated, the most predominant subject is then chosen. If it is the same as the channel, nothing happens, if it is not, a redirection message is sent.

You can find the code here :
https://github.com/PaulJurek/AIC_5101C_BERTRAND_IM-SAROEUN_JUREK